

Tarea #4: Pipeline Autónomo de Detección – Auto-etiquetado con Qwen VL, Transfer Learning y Validación

Tarea #4: Pipeline Autónomo de Detección – Auto-etiquetado con Qwen VL, Transfer Learning y Validación..... 1

1. Objetivo de la Tarea.....2

2. Instrucciones y Fases del Proyecto.....2

 Fase 1: Recolección y Auto-etiquetado con Qwen3.5 (100 imágenes de perros Huskies).....2

 Fase 2: Transfer Learning (Ajuste Fino de YOLOv8s).....2

 Fase 3: Deployment (Despliegue del Detector).....3

 Fase 4: Validación Híbrida en Cascada con el VLM (Comparativa 0.8B vs 2B).....3

3. Script de Ejemplo: Inferencia con Qwen VL y Conversión a Formato YOLOv8..... 3

4. Ejemplo de Salida Esperada (Archivo *.txt de Anotación).....5

5. Tabla a incluir en Reporte Técnico..... 5

6. Cuestionario de Análisis Crítico.....6

7. Entregables Requeridos..... 7

1. Objetivo de la Tarea

El equipo diseñará e implementará un pipeline moderno de visión computacional que resuelva el problema de escasez de datos mediante el uso de Modelos de Lenguaje y Visión (VLM) avanzados. El estudiante aprenderá a generar un dataset de forma 100% automática (Auto-labeling) utilizando un VLM generativo para extraer coordenadas espaciales, mapear dichas predicciones al formato estructurado de YOLOv8, realizar Transfer Learning sobre un detector ligero, y mitigar falsos positivos mediante un paso de validación en cascada con el mismo VLM.

2. Instrucciones y Fases del Proyecto

Fase 1: Recolección y Auto-etiquetado con Qwen3.5 (100 imágenes de perros Huskies)

- Deberá recolectar un conjunto de 100 imágenes sin etiquetar de internet. Seleccionar imágenes que tengan múltiples instancias (varios perros Huskies cada imagen).
- **Selección del VLM:** Se utilizará el modelo de arquitectura abierta **Qwen3.5 (pueden elegir la versión de 2B o de 4B)** y deberán correrlo en un ambiente local de su computadora usando la GPU de la laptop.
- Se debe diseñar un prompt específico (Ej: "Detect the 'husky dogs' in this image and return its bounding box coordinates.") para obligar al VLM a retornar la localización exacta del objeto. (O pueden pedir en el prompt que ya haga la normalización).
- El script deberá procesar la respuesta de Qwen (que por defecto entrega las cajas delimitadoras en su propio formato) y realizar la conversión matemática al formato estándar de YOLOv8 (class_id x_center y_center width height normalizado de 0 a 1). Se generará un archivo *.txt por cada imagen. O prueben que les de la normalización ya directa
- Dibujen los BB en cada imagen, vean que estén correctos.

Fase 2: Transfer Learning (Ajuste Fino de YOLOv8s s=small)

- Estructurar 70 imágenes y los archivos *.txt generados por Qwen VL en un archivo de configuración dataset.yaml.
- **Aumento de Datos (Data Augmentation):** Debido al bajo volumen de datos (70 imágenes), es recomendable configurar técnicas de aumento de datos en el archivo dataset.yaml o mediante hiperparámetros de entrenamiento de Ultralytics (ej. mosaic, mixup, hsv_h) para evitar el sobreajuste de la red.

- Entrenar un modelo YOLOv8s preentrenado (de Ultralytics) utilizando transferencia de conocimiento sobre estas 70 imágenes para que aprenda a detectar de forma autónoma la nueva clase. Use el modelo de Ultralytics para simplificar el transfer learning.
- Las otras 30 imágenes servirán para testing.
- Dejen las otras clases pre-entrenadas activas.

Fase 3: Deployment (Despliegue del Detector)

- Desarrollar un script de inferencia optimizado en Python (*.py) que cargue el modelo YOLOv8s entrenado y procese flujos de imágenes simulando producción.

Fase 4: Validación Híbrida en Cascada con el VLM (Comparativa 0.8B vs 2B)

- Cuando el YOLOv8s entrenado detecte la nueva clase, el script recortará la región de la caja delimitadora (Crop).
- Este recorte se enviará de inmediato al modelo Qwen VL con un prompt de confirmación binario (Ej: "Is this a husky dog inside this image crop? Answer only Yes or No."). Si el VLM confirma con un "Yes", la detección final se aprueba; de lo contrario, se descarta por falso positivo.
- **Requisito de Comparación:** En esta fase de validación, el equipo deberá ejecutar el pipeline utilizando **dos versiones diferentes del modelo Qwen VL: la versión de 0.8B y la versión de 2B**. Deberán documentar y comparar el mAP resultante y los tiempos de inferencia obtenidos con ambas versiones.

3. Script de Ejemplo: Inferencia con Qwen VL y Conversión a Formato YOLOv8

A continuación, se proporciona un script base actualizable según la fase y versión del modelo elegido **(OPCIONAL PUEDEN UTILIZAR SU PROPIO CÓDIGO):**

```
import re
import torch
from transformers import AutoProcessor, AutoModelForVision2Seq

# 1. Configurar y cargar el modelo Qwen VL cuantizado en INT4
# Nota: Actualice 'model_id' según la fase:
# - Fase 1: Usar versión 2B o 4B (ej. "Qwen/Qwen3.5-4B-VL")
# - Fase 4: Usar versión 0.8B y luego 2B para comparar (ej. "Qwen/Qwen3.5-0.8B-VL")
```

```

model_id = "Qwen/Qwen3.5-4B-VL"
processor = AutoProcessor.from_pretrained(model_id)
model = AutoModelForVision2Seq.from_pretrained(
    model_id,
    torch_dtype=torch.float16,
    device_map="auto"
)

```

2. Definir imagen y prompt de búsqueda

```

image_path = "imagen_ejemplo.jpg"
prompt = "Detect the 'huskies dog' in this image and provide the bounding boxes."

```

```

# [Nota: Aquí el equipo implementará el pipeline de pase de mensajes del procesador]
# Supongamos que el VLM responde con la cadena de texto estándar: "[230, 450, 680, 890]"
# Donde el formato es [ymin, xmin, ymax, xmax] en una escala de 0 a 1000.

```

```

def convert_qwen_to_yolo(qwen_box, class_id=0):
    ymin, xmin, ymax, xmax = qwen_box

    # Normalizar de escala 1000 a escala 0-1
    xmin_n = xmin / 1000.0
    ymin_n = ymin / 1000.0
    xmax_n = xmax / 1000.0
    ymax_n = ymax / 1000.0

    # Calcular ancho y alto
    width = xmax_n - xmin_n
    height = ymax_n - ymin_n

    # Calcular centro de la caja
    x_center = xmin_n + (width / 2.0)
    y_center = ymin_n + (height / 2.0)

    return f"{class_id} {x_center:.6f} {y_center:.6f} {width:.6f} {height:.6f}"

```

```

# Ejemplo de uso con una predicción simulada de Qwen
qwen_box_detected = [230, 450, 680, 890]
yolo_line = convert_qwen_to_yolo(qwen_box_detected, class_id=0)

```

```

# Guardar en archivo .txt homologado
with open("imagen_ejemplo.txt", "w") as f:
    f.write(yolo_line + "\n")
print("Línea guardada en formato YOLO:", yolo_line)

```

4. Ejemplo de Salida Esperada (Archivo *.txt de Anotación)

Para cada imagen procesada en la Fase 1, el equipo debe generar un archivo de texto con el mismo nombre de la imagen. El contenido estructurado bajo la norma de YOLOv8 debe lucir exactamente así:

```
0 0.670000 0.455000 0.440000 0.450000
```

5. Tabla a incluir en Reporte Técnico

El equipo deberá rellenar la siguiente matriz técnica comparativa dentro de su reporte final:

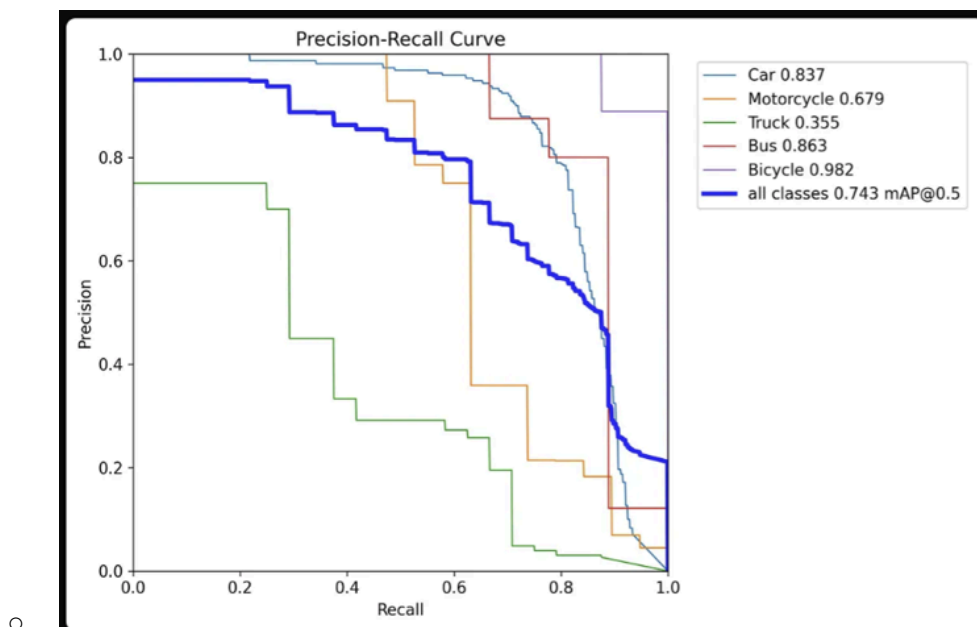
Recomendación: poner threshold bajo para que aumenten las detecciones.

Configuración del Pipeline	mAP@0.5	Falsos Positivos No-perros que dice que si son	Falsos Negativos Perros no detectados que si debería	Latencia de Inferencia (ms)	FPS Reales
YOLOv8s Ajustado (Solo)					
YOLOv8s + Qwen VL Validador (0.8B)					
YOLOv8s + Qwen VL					

Configuración del Pipeline	mAP@0.5	Falsos Positivos No-perros que dice que si son	Falsos Negativos Perros no detectados que si debería	Latencia de Inferencia (ms)	FPS Reales
Validador (2B)					

3 gráficas Precision-Recall @0.5 IoU - considerando:

- Detector solo
- Detector + validador 0.8B
- Detector + validador 2B



6. Cuestionario de Análisis Crítico

2. **Penalización de Recursos Computacionales:** Analice la degradación de FPS al activar Qwen VL como validador en cascada. Compare los resultados entre usar la versión de 0.8B y la de 2B. ¿Es viable este pipeline para un sistema de tiempo real crítico (ej. vehículos autónomos) o

se limita a aplicaciones asíncronas?

3. **Propagación del Error:** Si el VLM de auto-etiquetado (2B o 4B) comete un error sistemático de desalineación en las cajas durante la Fase 1, explique cómo impacta este sesgo matemático en la función de pérdida de regresión (Bounding Box Loss) de YOLOv8s durante el proceso de Transfer Learning.
 - ¿Consideran que es necesario que un humano revise las anotaciones automáticas?
4. **Sensibilidad del Prompt Engineering (VLM):** Basado en su experimentación, discuta el impacto de modificar ligeramente el prompt de auto-etiquetado. ¿Qué tanta varianza observaron en la consistencia de las coordenadas de salida? Analice si el prompt de validación binaria funcionó igual de bien para los modelos de 0.8B y 2B.
5. **Optimización Arquitectónica:** En la Fase 4, se utilizan modelos VLM (0.8B y 2B) para validar un recorte de imagen. Considerando principios de eficiencia computacional, argumente si sería más óptimo reemplazar esta segunda etapa por un clasificador convolucional ligero (ej. ResNet18 o MobileNet, etc) entrenado desde cero para confirmar si el recorte contiene al objeto. ¿Cuáles son las ventajas y desventajas frente al uso de VLM de 0.8B y 2B?

7. Entregables Requeridos

El equipo deberá entregar un archivo comprimido (o enlace a repositorio) que contenga estrictamente lo siguiente:

- **Código Fuente (Scripts estructurados):**
 - `auto_labeling.py`: Script de inferencia del VLM (Fase 1), procesamiento del prompt y generación de los archivos `.txt`.
 - `train_yolo.py` (o Notebook): Código para el proceso de Transfer Learning de YOLOv8.
 - `hybrid_inference.py`: Script del pipeline de validación en cascada implementando la selección de modelos de 0.8B y 2B (Fase 4).
- **Reporte Técnico (PDF):** Documento formal que incluya:
 - La matriz técnica comparativa completada (Sección 5).
 - **Gráficas de Entrenamiento:** Curvas de pérdida (Bounding Box Loss, Objectness Loss), curvas de Precisión, Recall y mAP@0.5.
 - **Matriz de Confusión:** Generada por el modelo sobre el conjunto de prueba (30 imágenes).
 - **Análisis de resultados visuales:** Ejemplos de detecciones exitosas, falsos positivos mitigados por ambas versiones de la cascada y casos de falla.
 - Respuestas fundamentadas al Cuestionario de Análisis Crítico (Sección 6).